Implemented using the STM32 HAL library (STM32CubeF1 HAL drivers)

Voltage Protector Device - Design Document

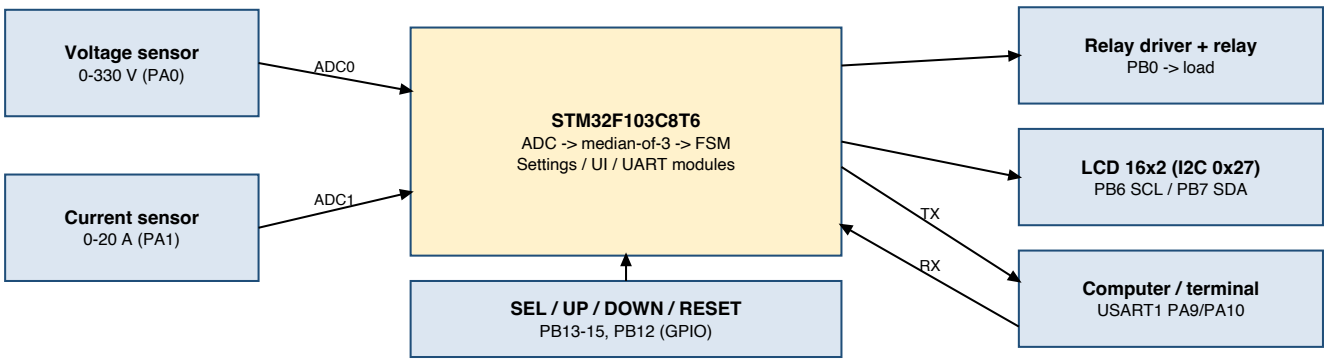
FSMB Recruitment 2026, Phase 1, Embedded System Engineer

1. What I built

A voltage protector on an STM32F103 Blue Pill. It keeps reading voltage and current, and switches a relay off if the voltage goes too high or too low, or the current goes too high. Limits can be changed with three buttons and an LCD on the device, or by commands from a PC over serial.

I only had the Wokwi simulator, so two potentiometers act as the sensors and an LED acts as the relay. Nothing was connected to mains. Receiving UART commands could not be verified in the simulator (section 8).

2. Block diagram



3. Parts and pins

Part	Pin	What it does
STM32F103C8 Blue Pill	-	Main controller, 64 MHz from the internal oscillator and PLL.
Potentiometer 1	PA0 (ADC0)	Fake voltage sensor, 0-330 V.
Potentiometer 2	PA1 (ADC1)	Fake current sensor, 0-20 A.
LED + 220 ohm	PB0	Relay. High = load connected.
RESET button	PB12	Clears a fault early. Active low.
SEL, UP, DOWN buttons	PB13, PB14, PB15	Pick a setting, raise it, lower it.
16x2 LCD, I2C backpack	PB6 SCL, PB7 SDA	Readings, state, setting being edited. Address 0x27.
USART1	PA9 TX, PA10 RX	Serial log and PC commands.
Onboard LED	PC13	Heartbeat: slow when fine, fast on fault.

The Blue Pill has a 12-bit ADC and a UART, and Wokwi has a model for it. The I2C backpack saves pins (2 wires instead of 6). A real product would use a transformer and divider for voltage, a Hall or shunt sensor for current, and a driver plus relay. I did not build that.

4. How the protection works



Every 100 ms the readings are compared with the limits (defaults 250 V, 170 V, 10 A). On a fault the relay goes off and the device stays in DISCONNECTED. When the reading is back in range, or RESET is pressed, it waits 3 s in COOLDOWN and reconnects. If the fault is still there it trips again.

There is no hysteresis; the cooldown is a blunt way to stop relay chatter at a limit. If several limits are broken, only the first found is reported (V high, V low, then current).

5. Reading the sensors

Each channel takes three ADC conversions and uses the middle one, so one bad read does not trip or hide a fault. If the ADC fails, the last good value is kept. volts = raw x 330 / 4095, amps = raw x 20 / 4095. This is a DC level, not true RMS, which a real mains product would need.

6. Communication

UART, 115200 8N1. The receive interrupt fills a 128-byte ring buffer and the main loop parses a line on CR or LF. A line like `[STATUS] VOLT:231.0V CURR:5.00A STATE:MONITORING FAULT:NONE OV:250V UV:170V OC:10.0A` goes out every second as a continuous log.

I2C to the LCD is bit-banged on PB6/PB7 with plain GPIO instead of the hardware peripheral. It is write-only and ignores ACK. The PCF8574 drives the HD44780 in 4-bit mode. Rows are redrawn only when their text changes.

Buttons and relay are plain GPIO. Buttons are read every 20 ms and acted on at the falling edge, which also debounces.

Command	What it does
SET_OV:230	Overvoltage limit. Above UV, at most 330 V.
SET_UV:180	Undervoltage limit. Above 0, below OV.
SET_OC:8	Overcurrent limit. Above 0, at most 20 A.
STATUS	Print the status line now.
RESET	Same as the reset button.
HELP	List the commands.

Bad values get [ERR] and the old limit stays; good ones reply [OK].

7. Buttons and display

The LCD shows voltage and current on line 1, and OK, TRIP + fault, or COOLDOWN on line 2. SEL cycles view, set OV, set UV, set OC. UP/DOWN change the value by 5 V or 0.5 A. A refused change shows LIMIT REACHED, and after 10 s idle it returns to the view. Same limit checks as UART. The firmware is one main loop (no RTOS) split into sensors, fsm, relay, settings, ui and uart_comm; the only interrupt is UART receive.

8. Testing

Test	What I did	Result
Normal	231 V, 5 A	Relay on
Overvoltage	280 V	Relay off, OVERVOLTAGE
Undervoltage	132 V	Relay off, UNDERVOLTAGE
Overcurrent	15 A	Relay off, OVERCURRENT
Cooldown	Overvoltage, then 231 V	COOLDOWN, then relay on
Reset button	Pressed in fault	Goes to COOLDOWN
Response time	Overvoltage at 5010 ms	Relay low at ~5044 ms (34 ms)
LCD settings	SEL, UP, UP on OV	Display showed 260 V
UART commands	SET_*, HELP, STATUS	**Failed in simulator**

Serial output and logging work, and button settings show on the LCD. Commands typed into Wokwi never reached the chip. I tried wokwi-cli `write-serial`, interactive mode and the VS Code extension. In gdb, USART1 was configured correctly but the ring buffer never got a byte; bytes I placed there by hand were parsed fine. I think it is a simulator limit, but cannot rule out a bug of mine. It needs a real USB-serial adapter. Logs are in test_evidence/.

9. Missing or could be better

- UART commands unverified, as above.
- Limits are RAM only and reset to defaults on restart.
- Faults are polled every 100 ms. No ADC watchdog interrupt and no hardware watchdog.
- At power-up the relay turns on about 1.4 ms after reset, before the first sample.
- No RMS measurement and no hysteresis.

10. Build and run

Needs arm-none-eabi-gcc, CMake 3.22+ and Ninja. In `src/`: `cmake --preset Debug`, then `cmake --build --preset Debug`. Open `project_files/` in VS Code with the Wokwi extension and start the simulator. See readme.md.